Презентация проекта

Корпоративный мессенджер с групповыми чатами

Обзор проекта

Цель проекта:

- Разработка масштабируемой корпоративной системы обмена сообщениями с поддержкой личных и групповых чатов, каналов, медиа-контента и интеграций

Ключевые характеристики:

- Микросервисная архитектура
- Масштабируемость до 5000 DAU
- Доступность 99.9%
- Интеграция с корпоративным LDAP
- Поддержка мобильных, веб и десктоп клиентов

Технологический стек:

- Backend: Java/Spring Boot, gRPC, REST
- Frontend: React, React Native
- Infrastructure: Kubernetes, Docker, Ansible
- Data: PostgreSQL, Redis, Elasticsearch, Kafka
- Storage: S3/MinIO

Основные возможности

Личные сообщения:

- Обмен сообщениями в реальном времени
- Отложенная доставка для офлайн пользователей
- Уведомления о прочтении
- Поиск по истории переписки
- Шифрование на транспортном уровне

Групповое взаимодействие:

- Создание каналов (публичных и приватных)
- Создание групповых чатов
- Треды сообщений
- Реакции на сообщения
- Администрирование (бан, удаление, назначение админов)

Дополнительные функции:

- Поддержка медиа (картинки, видео, документы до 2 ГБ)
- Упоминания пользователей (@username), хэштеги
- Пиннинг и избранное
- Боты и интеграции

Концептуальная архитектура - компоненты системы

Периметр:

- Load Balancer
- API Gateway (входная точка)

Микросервисы:

- Auth Service аутентификация и авторизация / User Service - управление профилями
- Chat Service - личные чаты / Group Chat Service - каналы и группы
- Media Service - хранение и доставка медиа
- Notification Service - уведомления / Search Service - полнотекстовый поиск

Инфраструктура:

- Kafka - брокер сообщений
- PostgreSQL - реляционные данные
- Redis - кэш и сессии
- Elasticsearch - поиск
- S3/MinIO - хранение медиа

Observability:

- Prometheus + Grafana - мониторинг
- Opensearch - логирование
- Jaeger/Tempo - трассировка

Архитектурные паттерны
Ключевые решения

CQRS (Command Query Responsibility Segregation):

- Разделение операций чтения и записи
- Command Path: запись в PostgreSQL Primary
- Query Path: чтение из PostgreSQL Read Replicas
- Оптимизация для высокого соотношения чтения/запись

Event Sourcing:

- История сообщений как последовательность событий
- События: MessageCreated, MessageEdited, MessageDeleted
- Восстановление состояния через воспроизведение событий

CDC (Change Data Capture):

- Debezium отслеживает изменения в PostgreSQL WAL
- Публикация изменений в Kafka topic `pg.changes`
- Обновление read models в реальном времени

Асинхронная обработка:

- Уведомления через Kafka
- Индексация в Elasticsearch
- Обработка медиа
- Аналитика и логирование

Безопасность и управление доступом

Аутентификация:

- Интеграция с корпоративным LDAP (Active Directory)
- Двухфакторная аутентификация (OTP)
- JWT токены (Access: 1 час, Refresh: 30 дней)
- mTLS для коммуникации между сервисами

Авторизация:

- RBAC (Role-Based Access Control) - Роли: Admin, Moderator, User
- Проверка прав на API Gateway и сервисах

Шифрование:

- TLS 1.3 для всех соединений
- mTLS между сервисами
- Шифрование данных в покое (PostgreSQL, S3, Redis)

Защита от атак:

- Rate Limiting (100 req/sec на IP)
- Параметризованные запросы к БД
- Валидация входных данных
- Мониторинг безопасности

Хранение секретов:

- HashiCorp Vault, автоматическое внедрение в Kubernetes Pods

Масштабируемость и отказоустойчивость

Масштабируемость:

- Горизонтальное масштабирование всех сервисов kubernetes HPA (Horizontal Pod Autoscaler)
- Шардинг PostgreSQL по user_id
- Redis Cluster (3+ ноды)
- Elasticsearch кластер с шардингом
- Kafka кластер (3+ брокера)

Отказоустойчивость:

- Circuit Breaker для защиты каскадных сбоев
- Retry с экспоненциальной задержкой
- Connection Pooling (PgBouncer)
- Streaming Replication PostgreSQL

Механизмы защиты:

- Rate Limiter на API Gateway
- DLQ (Dead Letter Queue) для Kafka
- Health checks и readiness probes- Автоматическое восстановление подов
- **Целевые показатели:** Доступность: 99.9% / Задержка сообщений: < 500 мс / Масштабируемость: 5000 DAU / RPO: 5-30 минут / RTO: 4 часа

Стратегия развертывания Infrastructure as Code

Инструменты:

- Ansible базовая инфраструктура
- Helm - управление приложениями в Kubernetes
- Kustomize - кастомизация конфигураций
- ArgoCD - GitOps подход

Развертывание инфраструктуры (Ansible):

- Установка Kubernetes кластера / Настройка сетевых правил и firewall / Установка PostgreSQL и Redis (вне Kubernetes) / Установка MinIO, Vault, Prometheus, Grafana, Opensearch

Развертывание приложений (Helm):

- Каждый микросервис в отдельном Helm chart
- Версионирование Helm charts
- Отдельные конфигурации для dev/staging/prod

Окружения:

- Dev - разработка, быстрые итерации
- Staging - предпродакшн, полное тестирование
- Production - продакшн, высокая доступность

*PostgreSQL и Redis развертываются вне Kubernetes на отдельных виртуальных машинах для обеспечения стабильности и производительности.

CI/CD процесс

CI (Continuous Integration):

- Автоматическая сборка Docker образов
- Unit tests на каждом commit
- Integration tests на каждом PR
- Code Review для всех изменений
- Сканеры безопасности (SAST, DAST)

CD (Continuous Deployment):

- Автоматическое развертывание в dev при merge в master
- Автоматическое развертывание в staging
- Ручное подтверждение для production
- GitOps через ArgoCD

Стратегия обновлений:

- Rolling updates для stateless сервисов
- Canary deployments для критических изменений
- Blue-green deployments для крупных релизов
- Database migrations отдельно от приложения

Тестирование и Observability

Типы тестов:

- Unit tests - тестирование функций и классов (Junit)
- Integration tests - взаимодействие сервисов (Testcontainers)
- E2E tests - пользовательские сценарии (Cypress)- Contract tests - API контракты (Pact)
- Load tests - нагрузочное тестирование (K6, Gatling)

Нагрузочное тестирование:

- Базовая нагрузка: 1000 concurrent users
- Пиковая нагрузка: 5000 concurrent users
- Стресс-тестирование: 10000+ users
- Endurance testing: 24 часа

Мониторинг (Prometheus + Grafana):

- Business metrics: пользователи, сообщения, чаты
- Application metrics: request rate, error rate, latency
- Infrastructure metrics: CPU, memory, disk, network
- Kubernetes metrics: pods, nodes, deployments
- **Логирование (Opensearch)**: Структурированные логи в JSON / Correlation ID для трассировки / Уровни: DEBUG, INFO, WARN, ERROR
- **Трассировка (Jaeger/Tempo)**: Распределенная трассировка запросов / OpenTelemetry для сбора трасс / Визуализация пути через микросервисы

Итоги

Достигнуто:

- Спроектирована микросервисная архитектура
- Определены функциональные и нефункциональные требования
- Разработана схема безопасности и управления доступом
- Спроектирована стратегия CI/CD
- Определены инструменты мониторинга и логирования
- Подготовлена документация по развертыванию

Конец

- Спасибо за внимание